



CMX-FFS-THINTM

CMX Systems, Inc. provides a range of specialized software designed for the development of file systems on embedded devices.

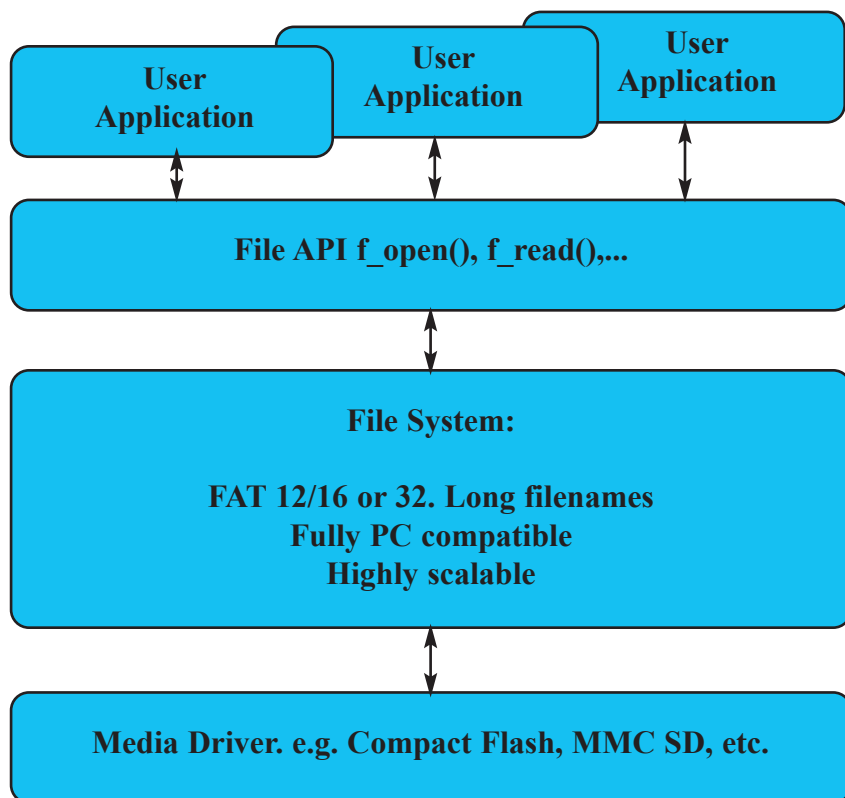
CMX-FFS-THIN is a full-featured file system targeted at embedded device developers - with limited resources available to them - who wish to add devices to their products that require a FAT12/16/32 compliant media to be attached to them. The code has been highly optimized for both speed and footprint to allow the developers to extract the most out of their system with a minimum of effort. Typical usage includes adding a Compact Flash card or a Multimedia card to a device. The system includes pre-tested sample drivers for ease of development. The API is standard and easy to use - sample applications in source code are provided to ensure quick integration. Hardware reference designs are also available.

Key Features

- **FAT12/16/32**
- **Long FileNames**
- **ANSI C compliant**
- **Standard API (fopen(), fread() etc.)**
- **Highly Scalable**
- **Reduced Footprint**
- **Multiple Simultaneous Files Open**
- **Handles Media Errors**
- **RTOS Independent**
- **Fully Integrated with CMX RTOSes and TCP/IP stacks**
- **Robust - Re-entrant**
- **Royalty Free Source**
- **Sample Applications**
- **Comprehensive Programmer's Guide**

Footprint

Typical Values	8-bit		16-bit	
	Code	Data Stack	Code	Data Stack
Read Only	<4K	<2K <0.5K	<6K	<2K <0.5K
Read/Write	<6K	<2K <0.5K	<10K	<2K <0.5K
Complete	16K	<2K <0.5K	20K	<2K <0.5K



CMX-FFS-THIN Software Structure

CMX-FFS-THIN is a derivative of the highly successful CMX-FFS-FAT system. CMX-FFS-THIN has been compiled and verified with many development environments to guarantee the highest possible level of portability. The User implementation guide contains detailed information as to how to port the system. CMX-FFS-THIN is almost agnostic as regards operating system. Also provided is a RAM drive which enables a system to be run independently of any hardware constraints, such as the availability of a new hardware interface. Typically, customers have a file system up and running within hours of receiving the system!

CMX-FFS-THIN includes a wealth of build options to enable developers to build exactly the system they need and no more.

Hardware Design and Implementation

CMX Systems, Inc. has a variety of hardware reference designs available for different hardware platforms and for different peripheral types. These designs are for 8-bit to 32-bit platforms. For a particular environment please contact CMX directly.

Application Programmer's Interface

A full suite of API functions is provided. The routines are entirely standard except that their names have been changed to avoid library conflicts, i.e., `fopen()` has become `f_open()`. These routines allow easy initialization and management of multiple volumes, directories and long filenames with complete wild card support.

The complete list of API functions is provided below.

<code>f_getversion()</code>	<code>f_mkdir()</code>	<code>f_settimedate()</code>	<code>f_seek()</code>
<code>f_initvolume()</code>	<code>f_chdir()</code>	<code>f_gettimedate()</code>	<code>f_tell()</code>
<code>f_format()</code>	<code>f_rmdir()</code>	<code>f_setattr()</code>	<code>f_eof()</code>
<code>f_hardformat()</code>	<code>f_rename()</code>	<code>f_getattr()</code>	<code>f_rewind()</code>
<code>f_getfreespace()</code>	<code>f_delete()</code>	<code>f_open()</code>	<code>f_putc()</code>
<code>f_setlabel()</code>	<code>f_filelength()</code>	<code>f_close()</code>	<code>f_getc()</code>
<code>f_getlabel()</code>	<code>f_findfirst()</code>	<code>f_write()</code>	
<code>f_getcwd()</code>	<code>f_findnext()</code>	<code>f_read()</code>	

Driver Interface

The interface between the file system and the physical devices attached is through the driver interface. This interface is simple and straightforward to implement - it is complemented by a set of sample drivers and detailed documentation.

The functions that must be implemented in any driver are listed below:

- GetPhy()** - Called by the file system to get information about the device attached (e.g., number of sectors, FAT type etc.)
- GetStatus()** - Called by the file system to get the current status of the physical device (e.g., card removed, card changed). For fixed devices this function maybe omitted.
- ReadSector()** - Called by the file system to read the specified sector of the attached device.
- WriteSector()** - Called by the file system to write data to the specified sector.

Also provide is a set of tested sample drivers to make porting as straightforward as possible. These include:

Compact Flash Card Supporting True IDE mode/ATA/PC Memory IO mode - with/without interrupts

MultiMediaCard/SD Card SPI mode (either hardware or software driven) and MMC mode

RAM Drive Allows simple creation of a driver in RAM - very useful for initial testing of the system and where full h/w interface is not yet available

For drivers not listed please contact cmx@cmx.com.